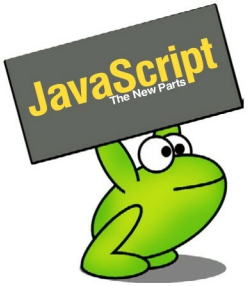

This is the third part in this series of articles and is about learning the new features introduced in JavaScript as part of the ECMAScript 2015 specification. Let's look at these new features — computed property keys, String helper functions and additions to Number objects.

In the second part of this series, we learned about binary/octal literals and enhanced object literals. We also looked at different levels of support in JS environments. Since September this year, an important event has happened—the merging of *Node.js* and *io.js*. *Node.js* is the community-led foundation and Joyent backed open source project. *io.js* is the community fork formed with the intention of having a truly community driven JS environment and bringing in ES6 features faster. Both groups have decided to merge and *Node.js version 4.0* is the resultant combination. For a detailed insight into the significance of the merger, read the article in the first reference given at the end of this article.

Something good about what we are going to learn this month is that these features are supported in most common JS environments, i.e., Node.js 4.0+, Babel, Chrome 45 and Firefox 41.

Computed property keys

In JavaScript, an object is used to group variables into a collection. Typically, the creation of an object involves defining a set of properties and assigning default values. In addition, JavaScript has the capability of dynamically (after definition) adding and deleting properties on-the-fly. The property names at the time of definition are strings, which follow rules of definition of a variable and are fixed, as in the example below:

```
let point = {
  x: 10,
  y: 20,
}
```

In ES6, JavaScript comes with the capability of defining property names as computed strings. This capability gives you programmatically defined property names that could change with each run and also adapt property names to the environment. Let us look at the various ways in which the property names can be computed:

```
1.  let i = 0;
2.  let std = "ecma";
3.  let fn = "print";
4.
5.  let obj = {
6.      ["js" + ++i]: i,
7.      ["js" + "2"]: ++i,
8.      ["js" + new Date().getFullYear()]: 10,
9.      [std]: 20,
10.     [fn]: function() { console.log("hello!"); },
11. };
12. console.log(JSON.stringify(obj)); // {"js1":1,"js2":2,"js
    2015":10,"ecma":20}
13. obj.print(); // hello!
```

In the above code snippet, lines 6 to 10 have different ways in which the property name is computed. In line 6, we have created a property name concatenating a string with a variable value. In line 7, we have concatenated two strings. In line 8, we have part of the property name as the return value of a function. With such capabilities, the program can have property names change at the time of execution. The same program, run in 2016, will give the property name *js2016*. In line 10, we have an example of a function name computed from a variable substitution.

String functions

There are new string functions that will be part of the JavaScript language with effect from the ES6 version. They can be categorised into template functions and search functions, as shown in Table 1.

Table 1

Category	Functions	Type
Template functions	raw~	Static
Search functions	*startsWith()*	*prototype*
	endsWith()	*prototype*
	includes()	*prototype*
	repeat()	*prototype*